

Games and Adversarial Search

★ Typical simple case for a game

✓ 1. Two-player game

- Only 2 players
- Example: Tic-Tac-Toe, Chess

↻ 2. Players alternate moves

- Not simultaneous
- Turn-based

Example:

Player 1 → Player 2 → Player 1 → Player 2

👉 This makes the problem structured like a **tree**

⚖ 3. Zero-sum game

This is **CRUCIAL**

One player's gain = other player's loss

Example:

- Win = +1 $\rightarrow$ opponent = -1
- Lose = -1 $\rightarrow$ opponent = +1

👉 No cooperation, pure competition.

4. Perfect Information

- **Both players see EVERYTHING**
- both players have access to complete information about the state of the game.
- No information is hidden from either player.
- No hidden cards, no secrets

Examples:

- ✓ Chess
- ✓ Tic-Tac-Toe
- ✗ Poker (hidden cards)

5. No chance (deterministic)

- **No randomness** (like dice)

Examples:

- ✓ Chess
 - ✗ Backgammon (dice involved) , Rock paper scissor (based on chance)
-
-

Can we use these search techniques?

Uninformed Search (BFS, DFS)

 NOT suitable

Why?

- These assume **no opponent**
- They **try to find a goal blindly**

BUT in games: The **opponent** is **actively trying to stop you**

Heuristic Search (A*)

 Also not directly usable

Why?

- A* assumes : "Environment is passive"

But here: **Opponent changes** the **future!**

Local Search

 Not suitable

Why? It improves a single state

- Doesn't consider **future opponent moves**

✗ Constraint Satisfaction (CSP)

👉 Not suitable

Why?

- CSP = satisfy constraints
 - Games = **strategic decisions over time**
-

🎮 How to Play a Game

At each step we evaluate a move, its score/outcome and check how feasible it is for us and if we should make it in a certain situation!

Step 1: Generate legal moves

From current state: **All possible actions you can take**

Example (Tic-Tac-Toe): Empty cells = possible moves

Step 2: Compute new states

Apply each move → generate new board

Step 3: Evaluate each state

- If game is finished → exact score
- Otherwise → **estimate using heuristic**

Step 4: Choose best move

- **Pick** the **move** that leads to **best outcome**

Step 5: Wait for opponent

- Opponent moves
- Repeat process



PROBLEMS

◆ 1. Representing the board (State Representation)

How do we store the game?

Example (Tic-Tac-Toe):

```
X | O | X
O | X | _
_ | _ | O
```

Can be stored as:

- 2D array
- String
- Matrix

◆ 2. Generating legal boards

Rules define: What moves are allowed

- On those basis we generate the next possible legal boards

Example: Empty cells only

3. Evaluating a position

 HARDEST PART

If game is over:

- Win = +1
- Lose = -1
- Draw = 0

If not:

- Use **heuristic** like:
 - **"Number of winning chances"**
 - **"Control of board"**
-

Evaluation Function (Core Idea)

An **evaluation function** (also called a **static evaluator**) is used to measure:

 ***"How good is this game position?"***

Because in most games (like Chess), we **can't search till the end** (too many possibilities), so we need a way to **estimate** who is winning.

- For a player who's trying to maximize its value, it will go towards the higher value. Whereas for the player who wants to minimize its value it will go to the lower value

Contrast with Heuristic Search

In earlier topics (like A*), evaluation meant:

- Estimate **distance to goal**

But here:

- There is **no fixed goal state**
- Instead, we evaluate:
 "How favorable is this position for me vs my opponent?"

So:

- Before → cost-based
- Now → **advantage-based**

Zero-Sum Property

Because games are **zero-sum**:

My gain = Your loss

👉 This allows us to use **ONE evaluation function** for both players.

Meaning of Values

Let's interpret the function $f(n)$:

- $f(n) \gg 0$ → 😊 Very good for me, terrible for opponent
- $f(n) \ll 0$ → 😞 Very bad for me, great for opponent
- $f(n) \approx 0$ → 😐 **Neutral** position
- $f(n) = +\infty$ → 🏆 I win
- $f(n) = -\infty$ → 💀 I lose

Evaluation Function Example (Tic-Tac-Toe)

Given:

$$f(n) = [\text{\# of 3-lengths open for me}] - [\text{\# of 3-lengths open for u}]$$

A **3-length** = any row, column, or diagonal of 3 cells

What does “open” mean?

👉 A line where you can still win

Example:

x | x | _

✅ Open for X (you can still win)

x | o | _

❌ Not open (blocked)

💡 Intuition

- More winning chances for you → good
- More for opponent → bad

👉 So we subtract

♟️ Alan Turing's Evaluation (Chess)

One of the earliest ideas:

$$f(n) = w(n) / b(n)$$

Where:

- $w(n)$ = total value of White's pieces
- $b(n)$ = total value of Black's pieces

🧠 Example Piece Values

Piece	Value
Pawn	1
Knight/Bishop	3
Rook	5
Queen	9

💡 Intuition

- If **ratio > 1** → **White** is **winning**
- If **ratio < 1** → **Black** is **winning**

General Form

Most evaluation functions look like this:

$$f(n) = w_1 * feat_1(n) + w_2 * feat_2(n) + ... + w_n * feat_n(n)$$

We combine:

- **Features** (things that matter)
- **Weights** (how important they are)

Features in Chess

Common features:

-  Piece count (material advantage)
-  Piece placement (good positions)
-  Squares controlled
-  King safety
-  Mobility (how many moves possible)

Example :

$$f(n) = 5 * (\text{material}) + 2 * (\text{mobility}) + 3 * (\text{king safety})$$

 Material matters more → higher weight

Real-World AI (Deep Blue)

The famous chess AI **Deep Blue** used:

👉 **8000+ features** 🤖

That means it considered:

- Tiny positional details
- Advanced patterns
- Strategic advantages

Why so many?

Because chess is:

- **Extremely complex**
 - **Small advantages matter a LOT**
-

That's Not How People Play

✗ Humans don't just look at the current board and decide

✓ Humans **think ahead**

When you play a game, you naturally think like:

"If I do this... then my opponent might do this... then I can do this..."

👉 This is exactly what AI tries to simulate.

The Process

- **Consider** your possible moves
- **Predict** opponent's responses
- **Predict** your next moves
- **Repeat...**

👉 This creates a **chain of possibilities** = **game tree**

Why Not Build the Full Game Tree?

Instead of full tree → build a **partial tree**

> building a complete tree is only possible for simple games!

Partial Game Tree

We only explore:

- **A few moves ahead**

Important Terms

- **Move** → **both players make a turn**
- **Ply** → **ONE player's turn**

What Do We Do With the Game Tree?

We use it to **choose** the **best move**

How? ➡ Using **Minimax algorithm** (coming next)

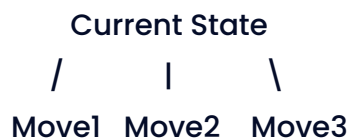
Game Trees

Games are modeled as **trees of possibilities**

Structure of a Game Tree

- **Nodes** → **board positions**
 - **Edges (arcs)** → **moves**
 - **Root** → **current state / current board configuration**
-

Example Idea



Each move leads to:

- **New board**
 - Opponent responses
-

Role of Evaluation Function

Static evaluator function rates a board position **$f(\text{board})$**

- **$f(\text{board}) > 0$** → good for us 😊
 - **$f(\text{board}) < 0$** → good for opponent 😬
-

MAX vs MIN Nodes

MAX Node (Your Turn)

- You want to **maximize score**
- Choose best possible move

MIN Node (Opponent's Turn)

- Opponent wants to **minimize your score**
-

Alternating Levels

- Nodes at level i are of the opposite kind from nodes at level $i + 1$

Game tree levels **alternate**:

Level 0 → MAX (you)

Level 1 → MIN (opponent)

Level 2 → MAX

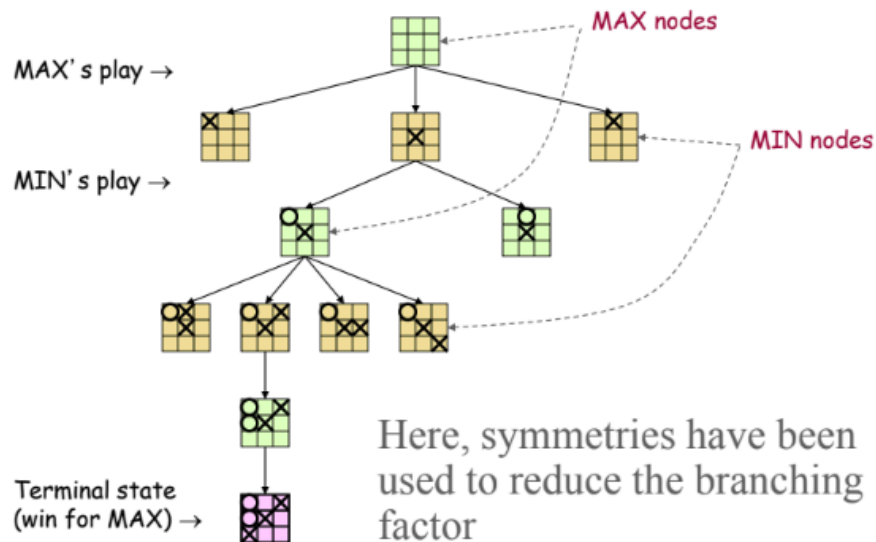
Level 3 → MIN



Rule

Every level **switches between MAX and MIN**

★🌳 Game Tree for Tic-Tac-Toe



- Each **node (square board)** = a game state
- Each **branch (line)** = a possible move
- The **top node (root)** = current board
- The tree expands with all possible moves

🏁 Terminal States (Leaves)

At the bottom:

👉 Terminal state (win for MAX)

- These are **final board positions**
- Game is over:
 - **Win / Lose / Draw**
 - **+1** → MAX wins
 - **-1** → MIN wins
 - **0** → Draw

› What does 'symmetries have been used to reduce the branching factor' mean?

In Tic-Tac-Toe, many moves are actually **equivalent**

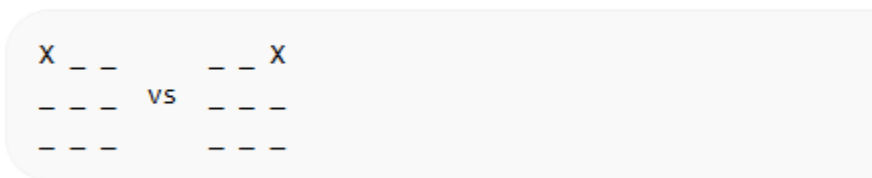
Example:

Placing X in:

- **Top-left** corner
- **Top-right** corner
- **Bottom-left** corner
- **Bottom-right** corner

👉 These are **basically the same** if you rotate the board

🧠 **Example**



👉 These positions are **symmetrical** (rotations of each other)

🌳 What is Branching Factor?

Number of children (moves) from a node

Example:

- First move in Tic-Tac-Toe = **9 choices** → **branching factor = 9**

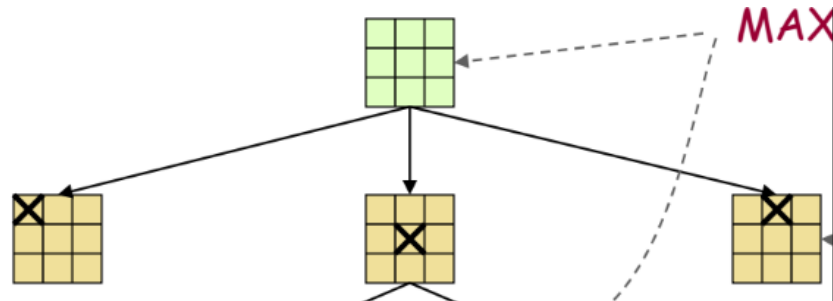
✂️ Reducing Branching Factor Using Symmetry

Instead of exploring ALL 9 moves, we may only explore 3!

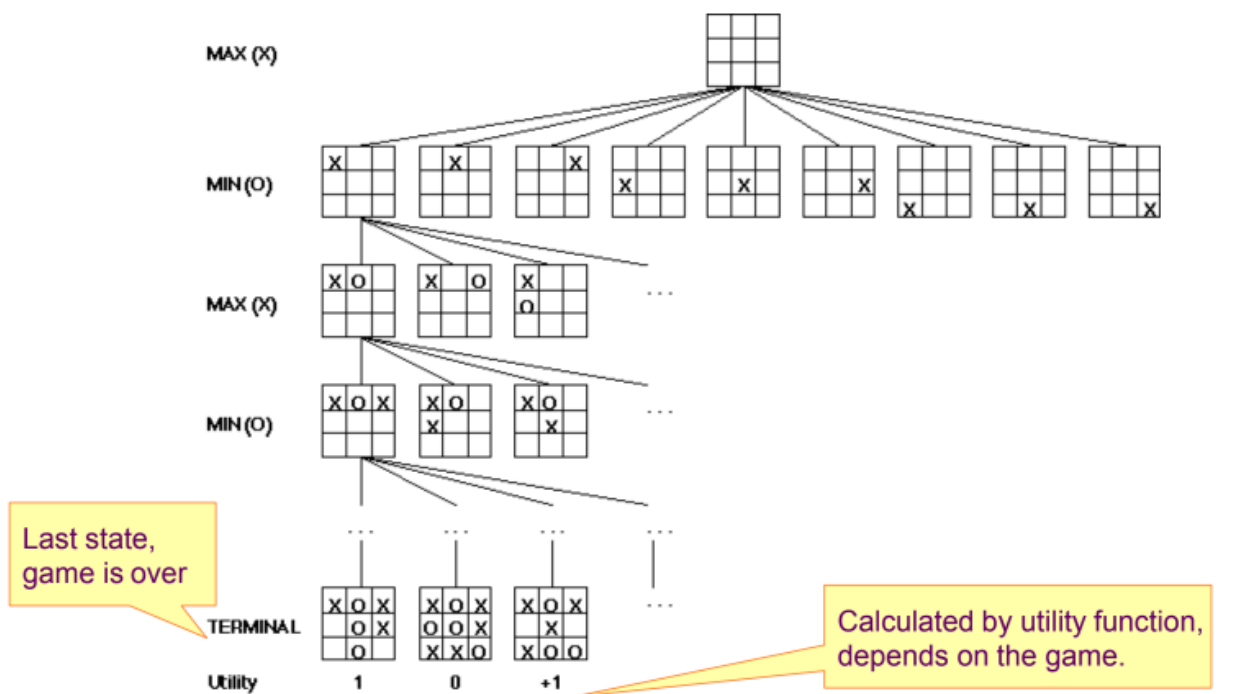
👉 We **group** similar (**symmetric**) moves

So we only explore:

- **One corner** move (represents all corners)
- **One edge** move (represents all edges)
- **Center** move



★ How to see the game as a tree



Two-Person Perfect Information Deterministic Game

Two players take turns

- Players move **one after another**
- No simultaneous actions

👉 This creates a **sequence of decisions**, which forms a **game tree**

Fully known board (Perfect Information)

- Both players can see:
 - Entire board
 - All previous moves



Examples

- ✓ Tic-Tac-Toe
- ✓ Chess
- ✗ Poker (hidden cards)

Deterministic moves

- No randomness involved
- **Same move** → **always same result**

Goal: Win (or tie)

- One player wins
- Or game ends in draw / tie

👉 This makes it a **zero-sum competition**

Key Objective

Find a **strategy** that **guarantees** the **best outcome**, assuming opponent plays **optimally**

Computer Games

Game = Search Problem

We treat the game like:

- States → board positions
- Actions → legal moves
- Goal → winning state

👉 So solving a game = **searching the best path**

Multi-Agent Environment

- Not just YOU making decisions
- There is another **agent (opponent)**

Conflict of Goals

- You want to win
- Opponent wants you to lose

👉 This is called: **Adversarial setting**

Nature of These Environments

✓ **Mostly deterministic + Fully observable**

It allows AI to:

- Build **accurate game trees**
 - Use **Minimax reasoning**
-

Why Games Are NOT Simple Search Problems

Branching Explosion

Example: Chess

- Average branching factor ≈ 35
- Each player plays ~ 50 moves

 **Total possibilities:**

$35^{100} \approx 10^{154}$ states 🤖

👉 This is **HUGE** (more than atoms in the universe!)

💡 **Meaning**

We cannot explore the full game tree

🕒 Finding Optimal Move (Real Challenge)

We want: **Best possible move within limited time**

❌ Problem

- Full search = impossible
 - Time is limited
-

✅ Solution

We:

- Search partially
 - Estimate the rest
-

🧠 Heuristic Evaluation Functions

Approximate how good a position is without exploring to the end

Idea

Instead of: Search until win/loss

We do: **Search few steps** → **evaluate** → **decide**

AI game playing works like this:

Current State



Generate Moves



Build Partial Game Tree



Use Evaluation Function at Leaves



Apply Minimax



Choose Best Move

Minimax

algorithm that tells you:

 ***"What move should I choose, assuming my opponent is playing perfectly?"***

Utility Function

First, we define a **utility (evaluation) function**:

- It measures:
 “How good is this board for Player 1 (MAX)?”

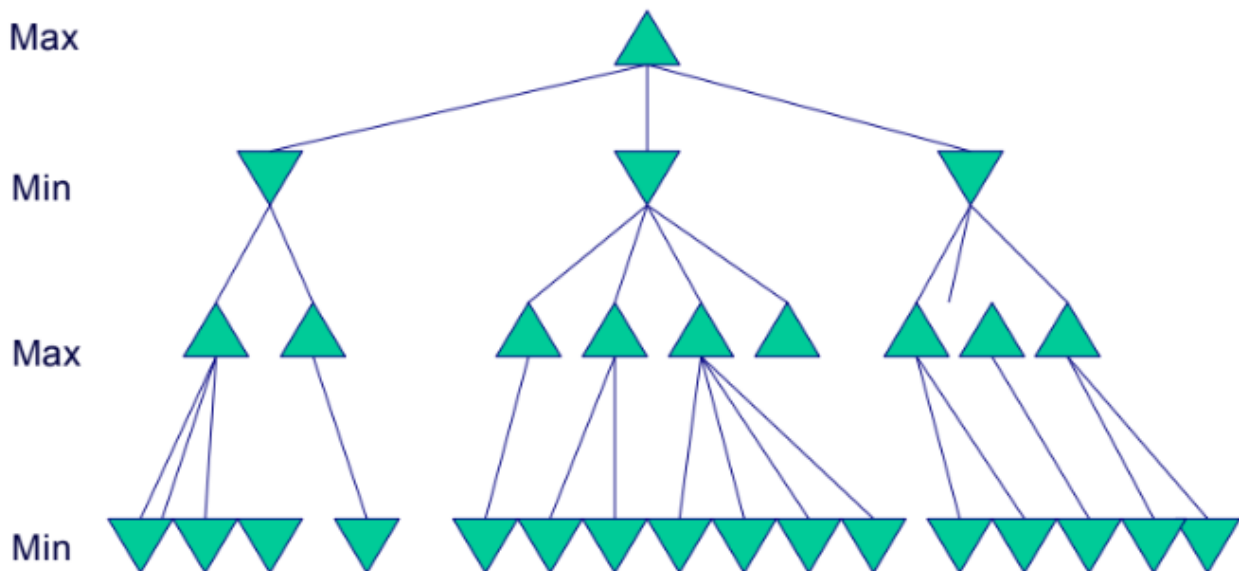
Roles

- **Player 1 (MAX)** → wants to **maximize** the score
 - **Player 2 (MIN)** → wants to **minimize** the score
-

Minimax Tree

We build a **game tree**:

- **Each level = one move**
- Tree grows **downward**
- Tree **alternates between “max”** (player 1 moves) and **“min”** (player 2 moves)



Minimax Tree Evaluation

Now comes the important part: **assigning values**

Leaf Nodes (Bottom of Tree)

We assign **utility values** to **leaves**

Case 1: Final State

assign the **maximum** or **minimum possible utility value**

If game is over:

- Win → **$+\infty$** (or **+1**)
 - Loss → **$-\infty$** (or **-1**)
 - Draw → **0**
-

Case 2: Non-final State

If not terminal:

 Use **evaluation function (heuristic)** to evaluate how good/bad the state is

Terminal Nodes

These are: **Nodes where evaluation function gives a value**

Either:

- **True outcome (win/loss/draw) OR estimated value**

Minimax Procedure (Step-by-Step)

Start

- **Root node = current board**
 - It is a **MAX** node
-

Expand Tree

- Generate all moves
 - Go down to certain **depth** (**ply** limit)
-

Evaluate Leaves

- **Apply evaluation function** at **bottom nodes**
-

Backpropagation (Backing Up Values)

Now we **move up the tree**

At MIN Nodes

- Choose **minimum value** from **children**

👉 Because opponent tries to hurt you

At MAX Nodes

- Choose **maximum** value from children

👉 Because you want best outcome

Example

MIN node children: [3, 5, 2] → picks 2

MAX node children: [3, 5, 2] → picks 5

Final Step (Decision)

Pick the **operator associated with** the **child node** whose **backed-up value** determined the value at the root

At root:

👉 Choose the move that leads to **highest value**

Minimax Theorem (Deep Idea)

Intuition

Assume **opponent** is **as smart as you (or smarter)**

Core Idea

- **Maximize** your **minimum gain**
 - **Minimize** your **maximum loss**
-

Simple Interpretation

"I will choose the move where even in the worst case... I'm still okay"

For every:

- Two-player
- Zero-sum game

👉 There exists:

- **A value V**
 - **Optimal strategies** for **both players**
-

Meaning

- Game has a **best possible outcome**
 - If both play perfectly:
 - **MAX gets V**
 - **MIN gets $-V$**
-

<https://www.youtube.com/watch?v=6ELUvkSkCts>

▲ means this node will get max values from below

▼ means this node will get min values from below



Minimax Tree Evaluation

For MAX player:



Generate Tree

- Go as deep as possible (within time)



Evaluate Leaves

- Use evaluation function



Back-Up Values

- MIN → choose minimum
- MAX → choose maximum



Final Decision

At root: 👉 Choose move with highest value

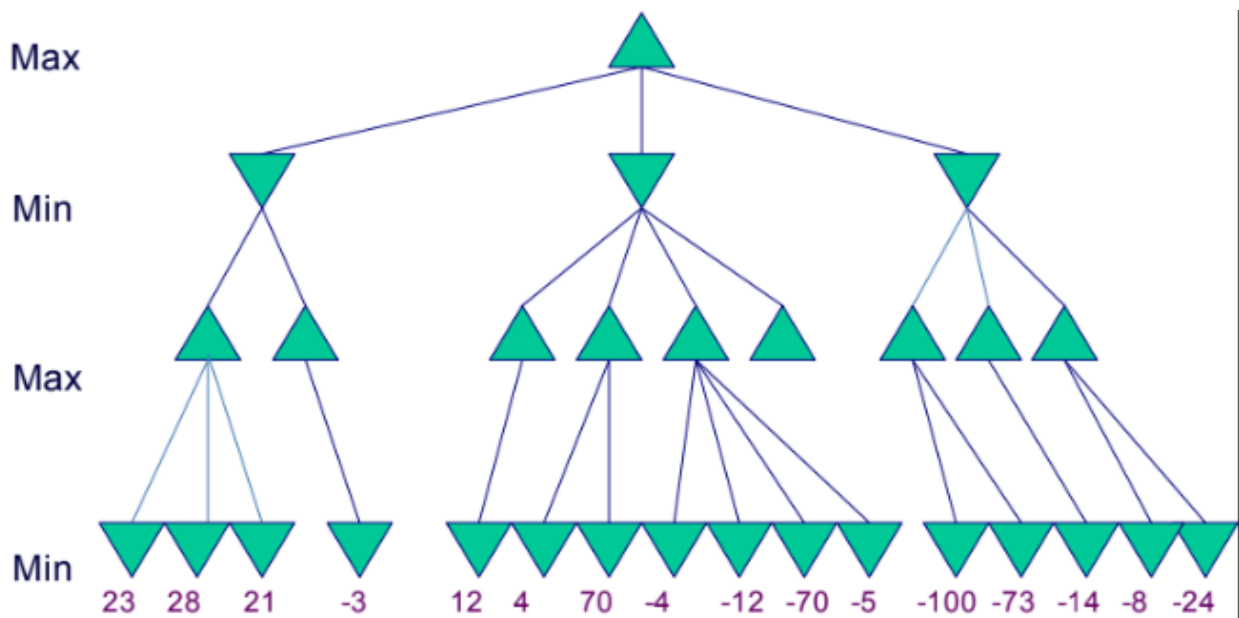
For the MAX player

1. Generate the game as deep as time permits
2. Apply the evaluation function to the leaf states
3. Back-up values
 - At MIN assign minimum payoff move
 - At MAX assign maximum payoff move
4. At root, MAX chooses the operator that led to the highest payoff

★ step by step working:

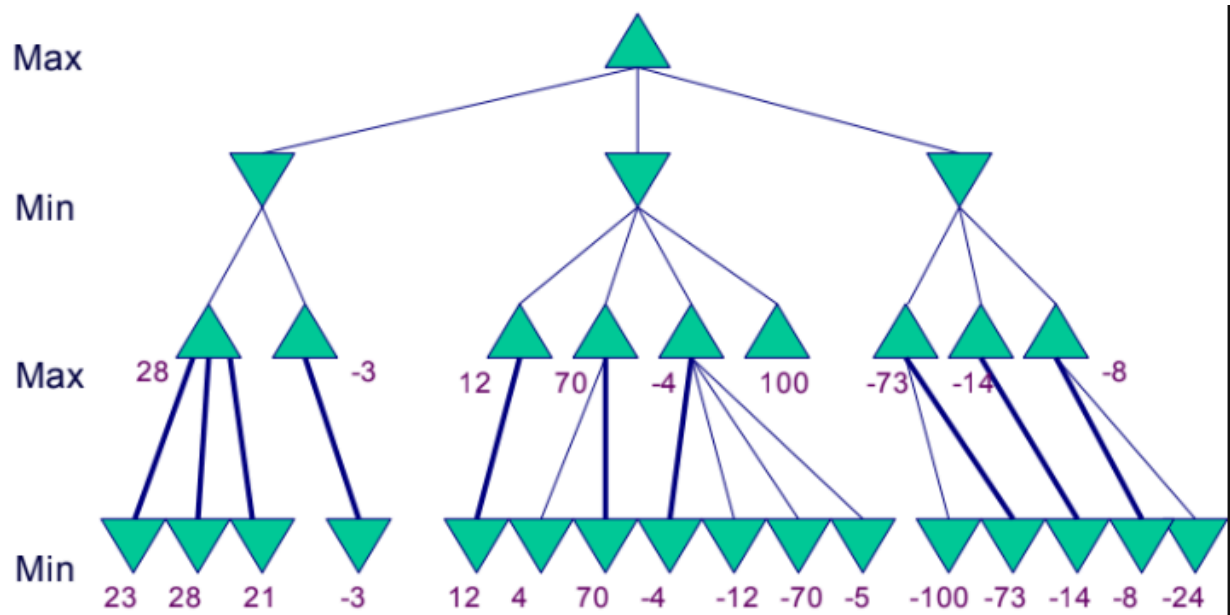
$minimax(s) =$

$$\begin{cases} utility(s) & \text{if } terminal(s) \\ \max_{a \in action(s)} minimax(result(s, a)) & \text{if } player(s) = MAX \\ \min_{a \in action(s)} minimax(result(s, a)) & \text{if } player(s) = MIN \end{cases}$$

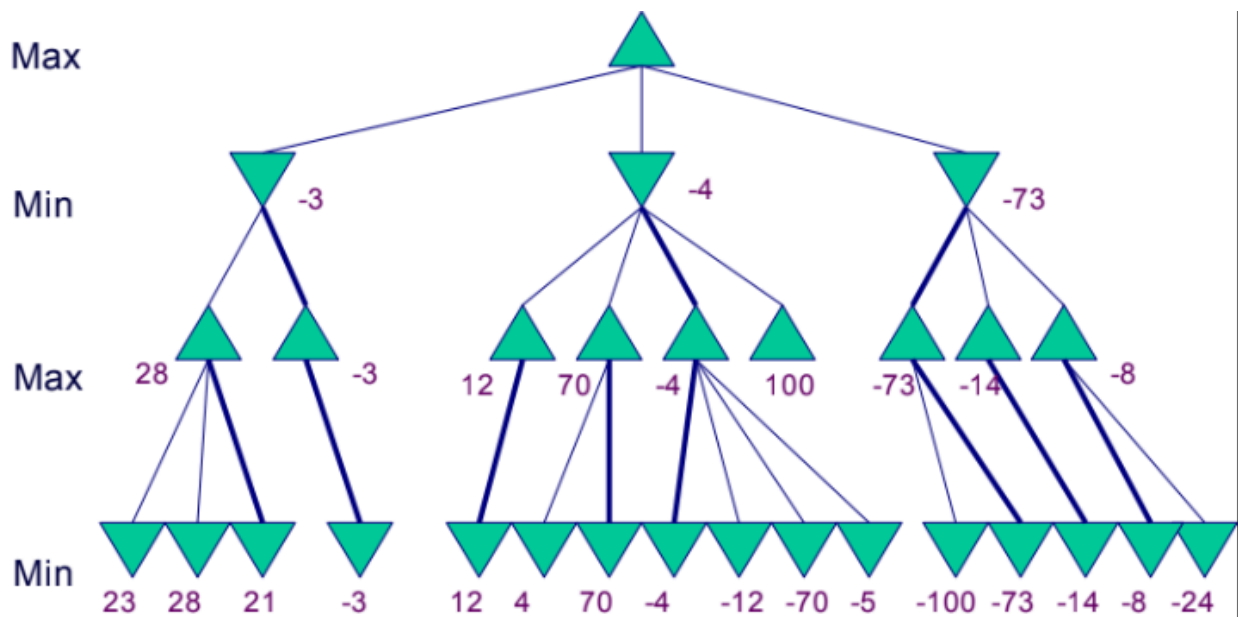


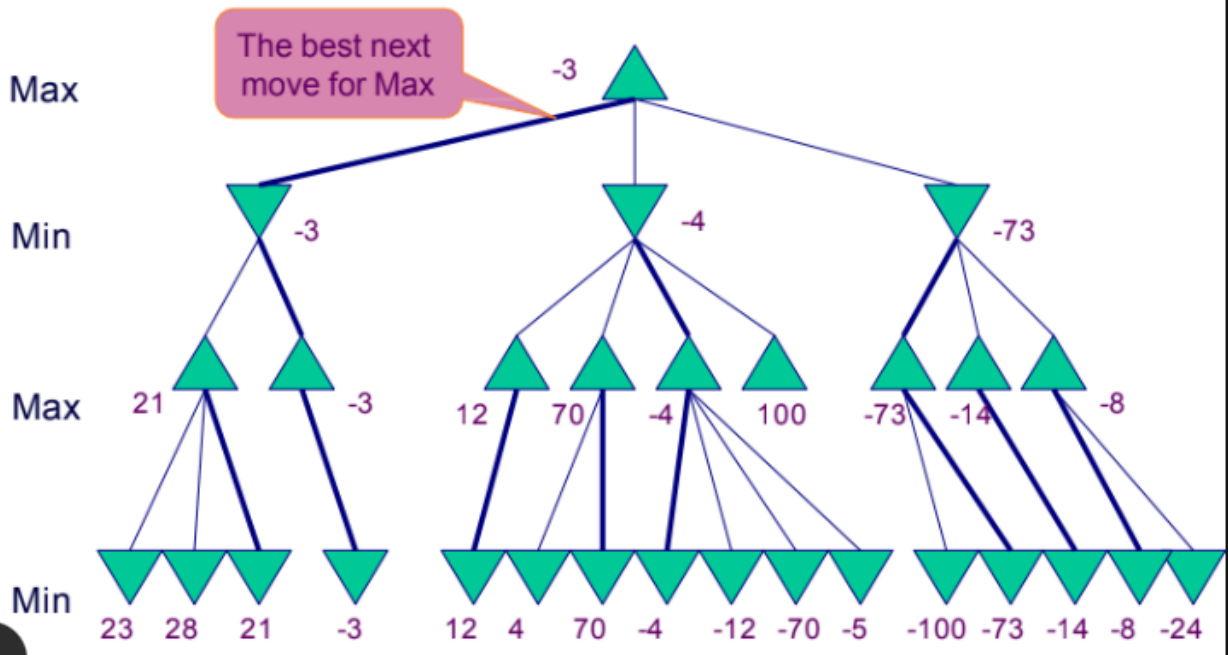
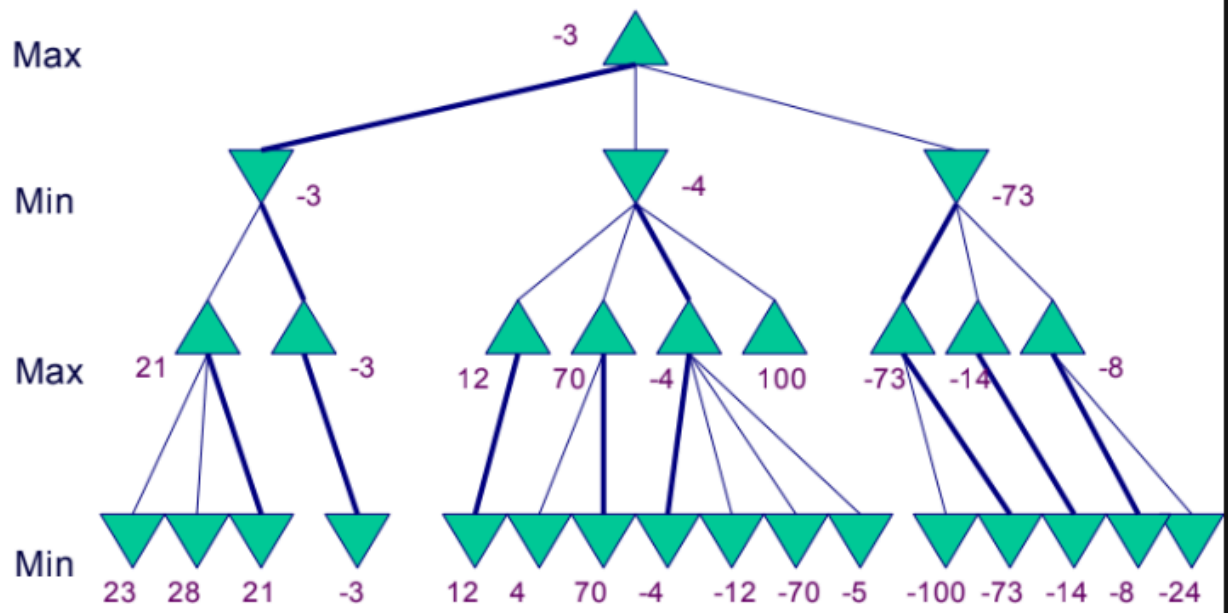
Terminal nodes: values calculated from the utility function

- Third level calls the terminal ones, they return their MAX values

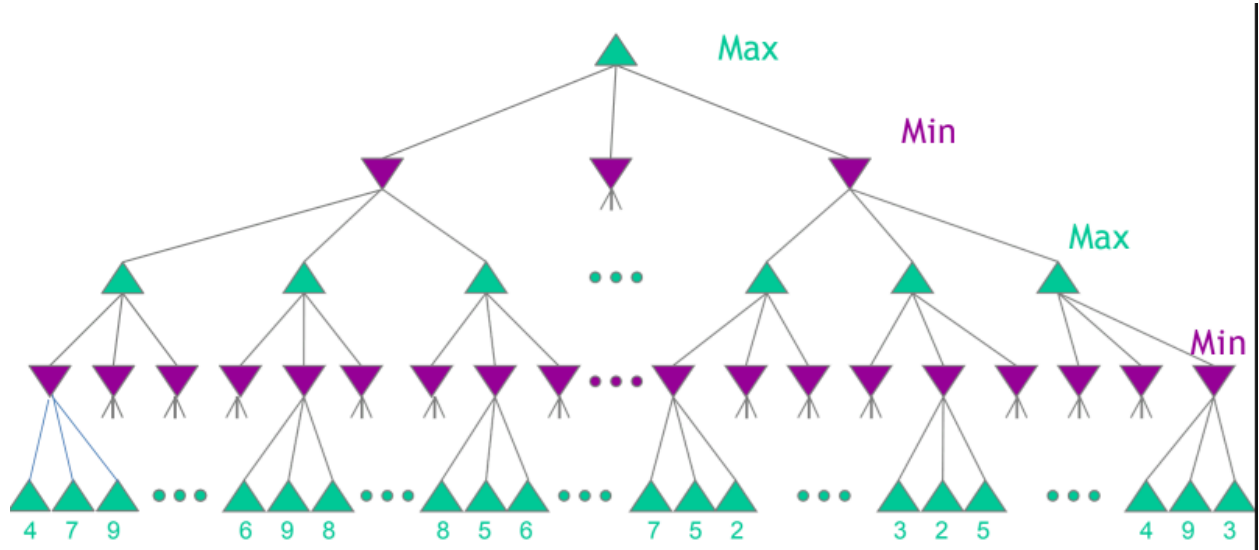


Other nodes: values calculated via minimax algorithm

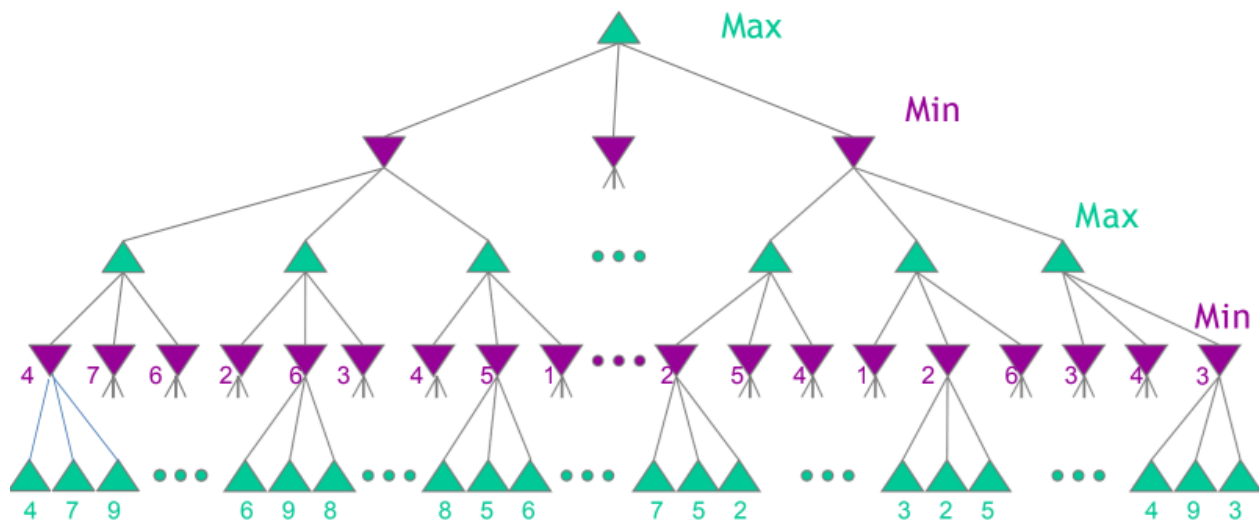


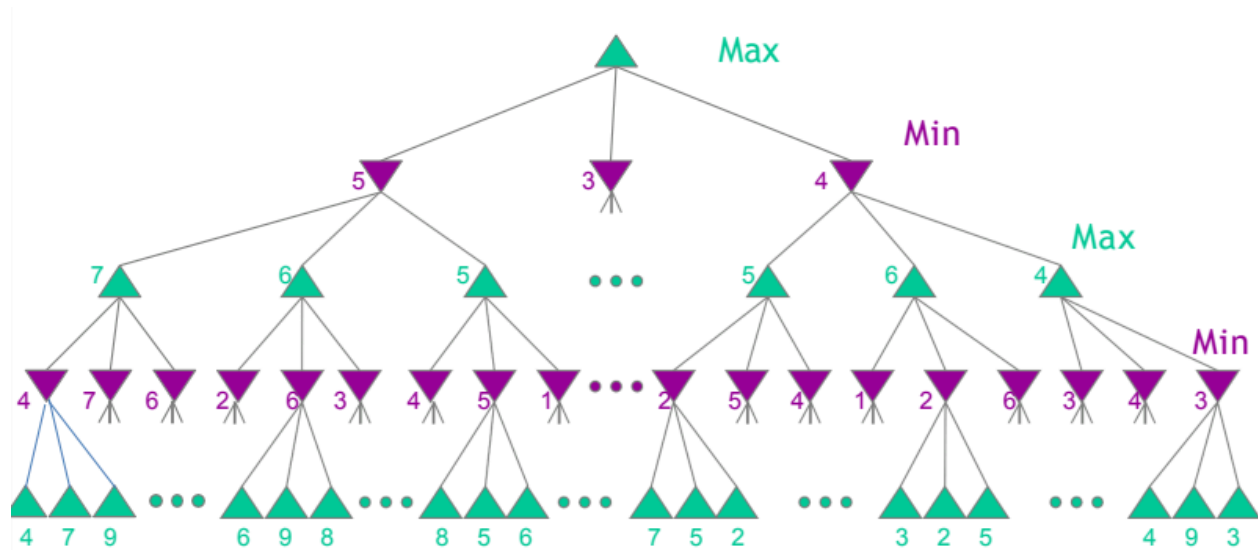
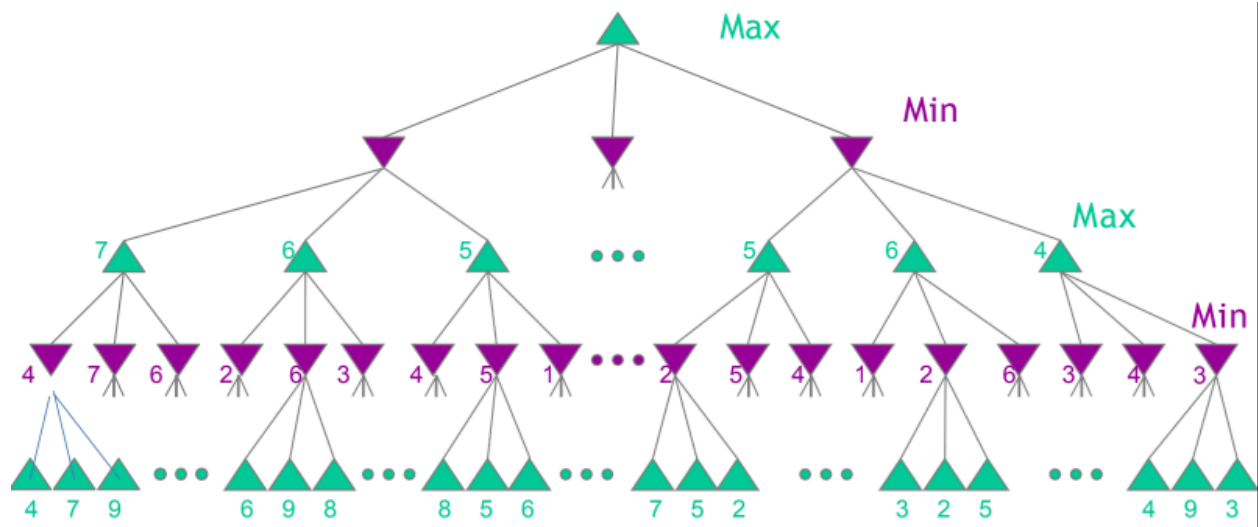


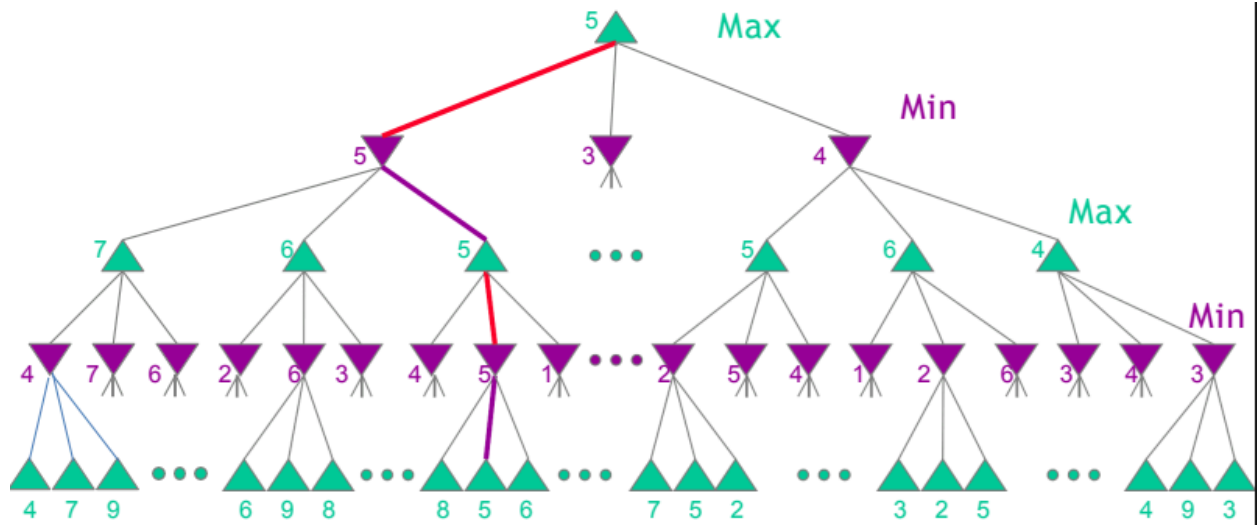
★ **example 2:**



^ get min value from terminals







Moves by Max and countermoves by Min



Properties of Minimax



Complete

✓ Yes (if the tree is finite)



Optimal

✓ Yes (against an optimal opponent)



Meaning

- If opponent also plays perfectly
 - 👉 Minimax gives the best possible move

💡 Important Insight

Minimax guarantees: **Best outcome in worst-case scenario**

👉 You can't do better than Minimax if opponent is perfect

🕒 Time Complexity

! $O(b^m)$

🧠 What do these mean?

- b = **branching factor** (number of moves per state)
- m = **depth** of the tree (**number** of **plies**)

😞 Why this is BAD

It grows **exponentially**

♟️ Chess Example

- $b \approx 35$
- $m \approx 100$

$35^{100} \approx 10^{154}$ 😱

👉 Completely **impossible to compute**

💾 Space Complexity

! $O(bm)$ (with **depth-first** search)

Meaning

- We don't store entire tree
- Only store:
 - **Current path**
 - **Some siblings**

Why smaller than time?

Because:

👉 Minimax uses **Depth-First Search (DFS)**

So memory = linear in depth

What determines depth?

- **Time available**
- Complexity of game
- Hardware power



Alpha-Beta Pruning

Alpha-Beta Pruning is an **improvement over Minimax**

👉 It gives: **Same result as Minimax, but MUCH faster**



Core Idea

Don't explore moves that will **never be chosen**



Why Do We Need It?

Minimax problem:

- Explores **ALL nodes**
- Very slow: **$O(b^m)$**

👉 Alpha-Beta:

- **Skips useless branches**
 - **Reduces computation**
-



Two Important Values



Alpha (α)

Best value MAX can guarantee so far

- Starts at $-\infty$
 - Gets **updated** when **MAX** finds **better** options
-



Beta (β)

Best value MIN can guarantee so far

- Starts at $+\infty$
 - Gets **updated** when **MIN** finds **better** options
-



Pruning Condition

We **stop exploring** when:

$$\alpha \geq \beta$$

👉 Meaning:

This branch cannot influence the final decision



What Does “Pruning” Mean?

👉 Cutting off a branch of the tree

👉 Not exploring it further



Your Slide Line Explained

“We don’t need to compute the value at this node.”

✓ Because:

- We already **found** a **better option elsewhere**
 - This node can **never beat it**
-



Key Line

“No matter what it is, it **can’t affect the root value**”

👉 That’s pruning!



Intuition with Example

Imagine:

- MAX already found a move with value = 10
- Now exploring another branch

At a MIN node:

- It finds a value = 5

What happens?

MIN will choose ≤ 5

But MAX already has 10

👉 So:

This branch is **useless** → **prune it**

α -Cut

If the **current max** value is **greater** than the **successors min value**, don't explore that **min subtree** any more.

- Happens at **MIN node**
 - Condition: $\beta \leq \alpha$
 - We **stop exploring MIN's subtree**
-

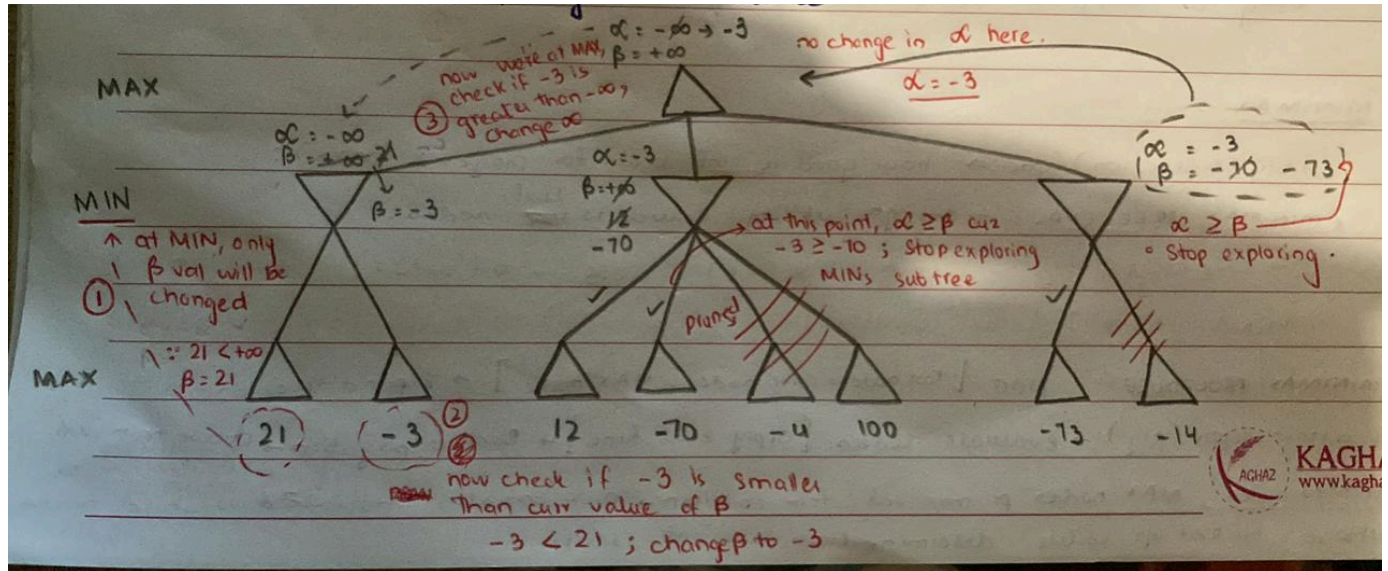
β -Cut

If the **current minimum** is **less** than the **successor's max value**, don't look down that **max tree** any more

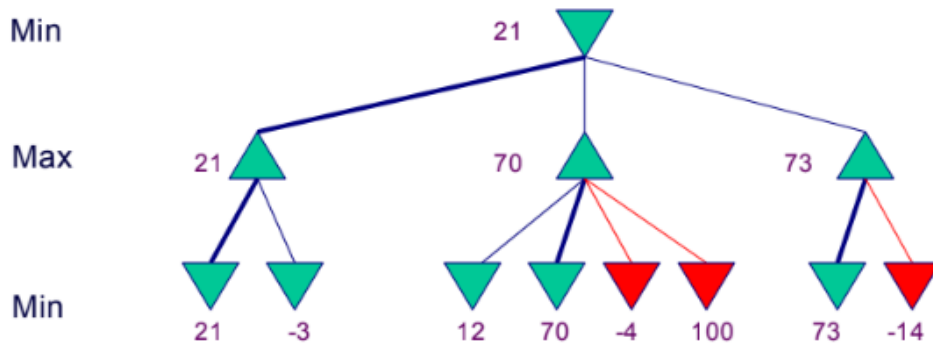
- Happens at **MAX node**
 - Condition: $\alpha \geq \beta$
 - We stop exploring **MAX's subtree**
-

> α -Cut Example: [37 - 46 slides]

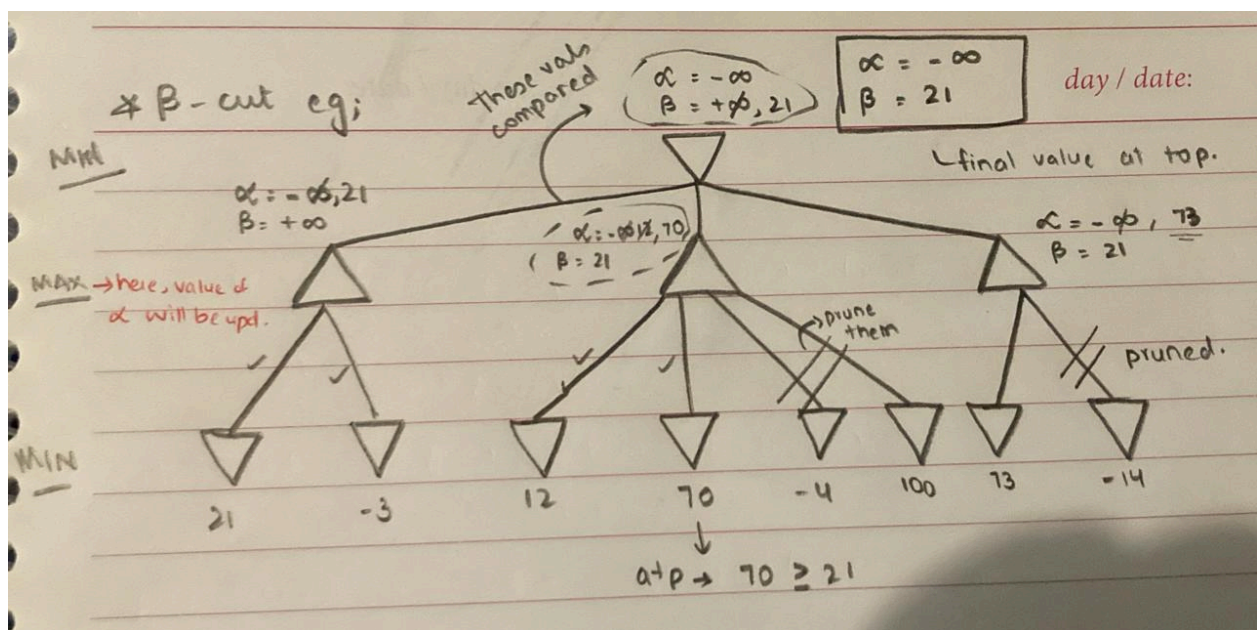
- DFS applied
- Leftmost $\rightarrow$ root $\rightarrow$ right node
- Initially:
 - **Alpha = -ve infinity**
 - **Beta = +ve infinity**



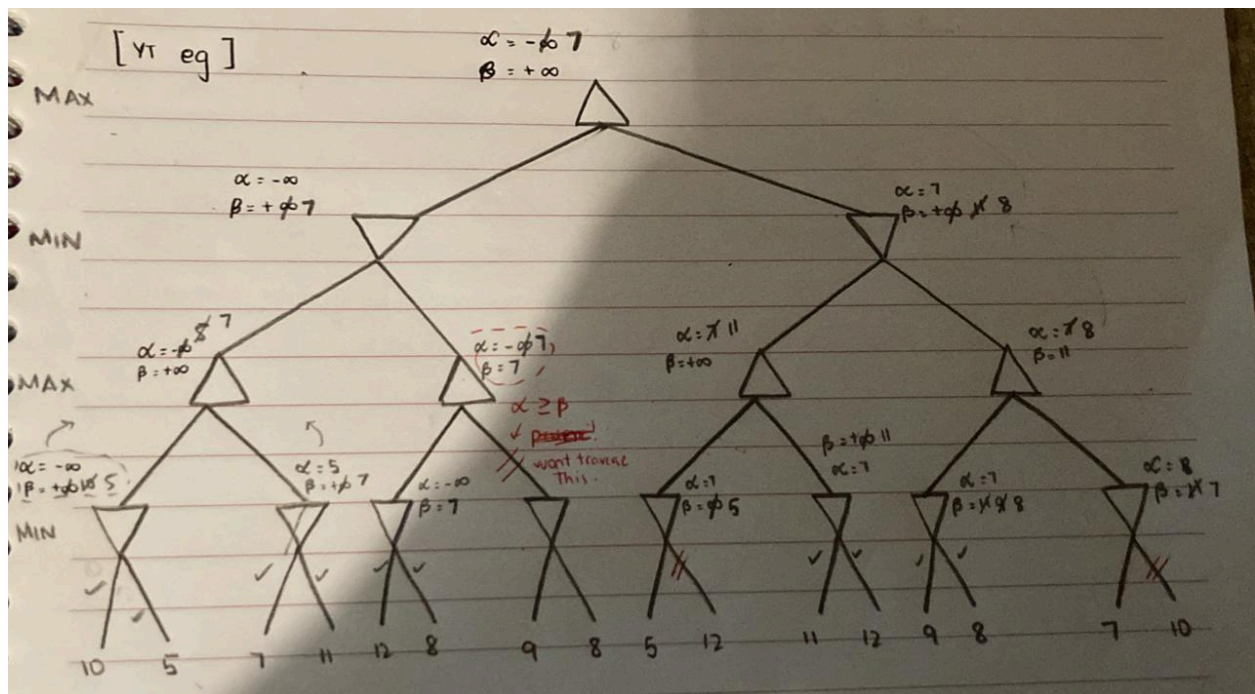
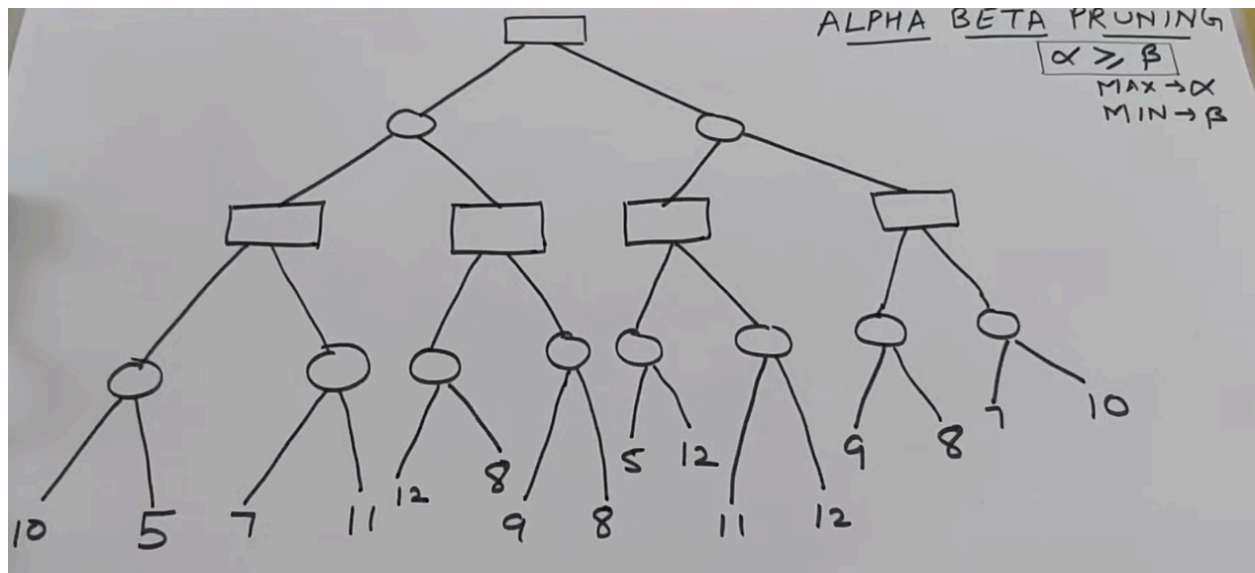
> β Cuts example



Some subtrees at second level already have values $>$ min from previous, so we can stop evaluating them.



https://www.youtube.com/watch?v=_i-lZcbWkps



› Alpha-Beta Example 2

[slide 49 - 57]

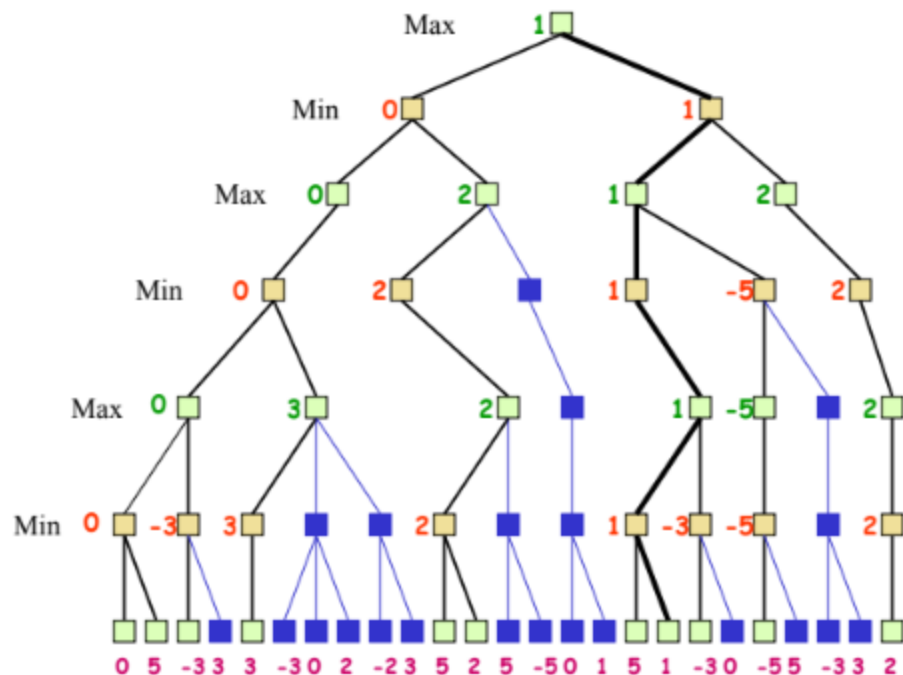
https://drive.google.com/file/d/1QtvXGPbJ-zKN8jMgX-sVWV7zaRJ_1PCr/view

› Alpha-Beta Tic-Tac-Toe Example 2

[slide 58 - 84]

https://drive.google.com/file/d/1QtvXGPbJ-zKN8jMgX-sVWV7zaRJ_IPCr/view

Blue wali means pruned / wont continue w that subtree further



★ $\alpha - \beta$ pruning

✂ Pruning Does NOT Affect Final Result

Even though we skip some branches:

👉 The **final move** chosen is **exactly the same** as **Minimax**

Because:

- We only prune branches that are **guaranteed to be useless**
- They **cannot influence the root decision**

Go Deeper in the Tree

“May allow you to go much deeper in tree”

Since we skip work:

- We save time
- So we **can search** more levels (**more plies**)

Deeper search = better decisions

Move Ordering

“**Good ordering** of **moves** can make **pruning much more efficient**”

What is Move Ordering?

The **order in which you explore** moves matters A LOT

Best Strategy

Explore the **best moves first**

Because:

- Good moves **update** α or β **quickly**
- This **causes more pruning later**

Perfect Ordering

“**Perfect ordering** reduces time to **$b^{(m/2)}$** ”

If you always check the **best move first**:

👉 Complexity improves from:

$$O(b^m) \rightarrow O(b^{(m/2)})$$

🤖 Huge Impact

“Doubles the depth you can search”

🧠 Storing Information Along the Path

α and β are not just local

👉 They are **carried down the tree**

📌 What we store

- α (alpha)
 - best MAX value found so far along this path
 - starts at $-\infty$
- β (beta)
 - best MIN value found so far along this path
 - starts at $+\infty$

💡 Why along the path?

Because:

- **Decisions earlier** in the tree **affect pruning later**
 - So we **pass α and β downward**
-

Pruning at MAX Node (More Precise Behavior)

What's new here

" α is updated using MAX of successors evaluated so far"

Meaning

At a MAX node:

- You keep updating α as you explore children
-

Pruning Condition (rephrased)

If value becomes $> \beta \rightarrow$ **stop**

Intuition

- MIN already has a better option (β)
 - So MAX exploring further is useless
-

If no pruning

- Return **maximum of all children**
-

Pruning at MIN Node (More Precise Behavior)

What's new here

“ β is updated using MIN of successors evaluated so far”

Meaning

At a MIN node:

- You keep updating β as you explore children
-

Pruning Condition (rephrased)

If value becomes $\less \alpha \rightarrow$ stop

Intuition

- MAX already has a better option (α)
 - So this branch is useless
-

If no pruning

- Return minimum of all children

Utility Evaluation Function

There is **no universal formula**

Each game has different:

- Rules
 - Strategies
 - Important features
-

Example

- Chess → pieces, king safety
- Tic-Tac-Toe → winning lines
- Checkers → piece positioning

👉 So evaluation must be **custom-designed**

“Stupid” Utility Function

Definition

+1 → Player 1 wins

-1 → Player 0/2 wins

0 → Draw

Why called “stupid”?

- It **only works at final states**
- Only **useful if** you **evaluate** the **entire tree**
- Gives **no information** for **intermediate** states

Assigning Numerical Values

We must **convert** a **board into** a **number**

👉 So Minimax can:

- **Compare states**
- **Choose best move**

⚠️ Important Note

More complex utility = harder comparison

Because:

- Minimax relies on **simple min/max operations**
- If evaluation is too complicated → harder to compute efficiently

Feature-Based Evaluation

Typically assign numerical values to lots of individual factors:

- **a** = piece difference
- **b** = queen advantage
- **c** = rook advantage

+ Combining Features

Linear Combination (MOST COMMON)

$$u = \alpha a + \beta b + \gamma c$$

- **Multiply** each **feature** by **importance (weight)**
- **Add** them together

› Why linear?

- Simple
- Fast
- Works well in practice

Other combinations

Possible but less common:

- Non-linear functions
 - Neural networks (modern AI)
-

What Makes a GOOD Utility Function?

Quality depends on:

- ♦ **Features chosen**
- ♦ **Feature scoring**
- ♦ **Weights**

Bad features or bad weights → bad AI decisions

Relative Value Matters

“**Absolute utility** value **doesn't matter** – **relative** value **does**”

Perfect Evaluation Function

? **What if evaluation is PERFECT?**

> You only need to search **1 level deep**

Perfect evaluation function is **impossible**

So we rely on:

👉 Approximation + deeper search

Evaluation Functions for Move Ordering

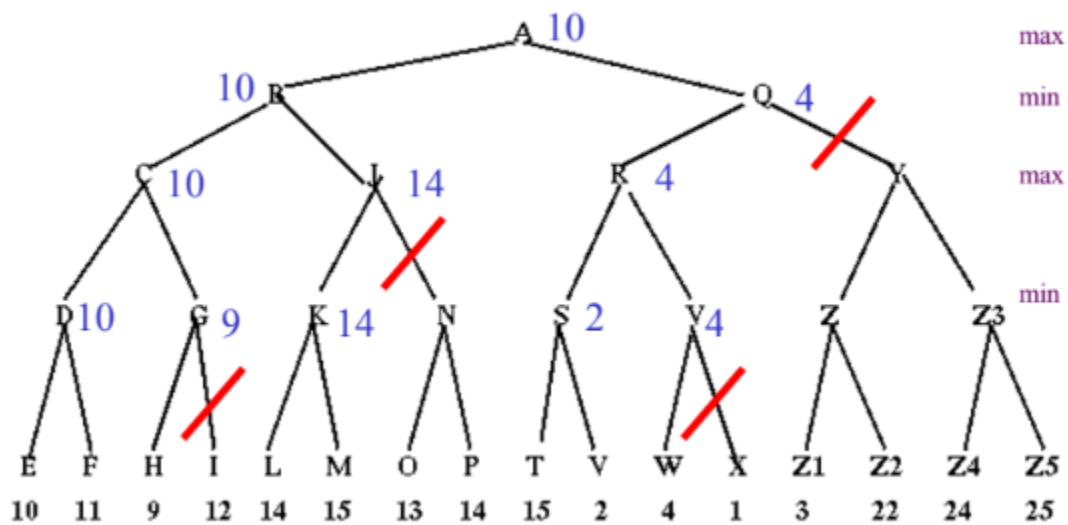
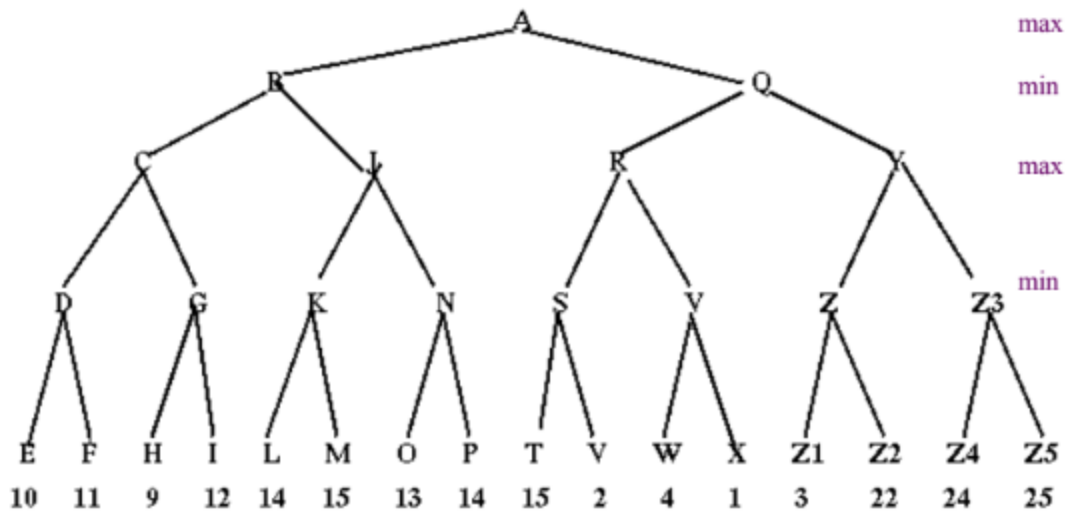
Use evaluation function to:

Decide which moves to explore FIRST

Process

- **Try** one **move**
- **Evaluate** quickly
- Order moves from:
 - Best → worst (for MAX)
 - Worst → best (for MIN)

★ Exercise [slide 97 onwards ...]



🎮 Checkers Case Study

- Initial board configuration

- Black
 - single on 20
 - single on 21
 - king on 31
- Red
 - single on 23
 - king on 22

- Evaluation function

$$E(s) = (5x_1 + x_2) - (5r_1 + r_2)$$

where

x_1 = black king advantage,

x_2 = black single advantage,

r_1 = red king advantage,

r_2 = red single advantage

